

DUOC UC

Escuela de Informática y Telecomunicaciones

Evaluación Final Transversal (EFT)

Predicción de churn con IA sobre un pipeline DataOps: proyecto completo, infraestructura, monitoreo, gobernanza y estrategia de despliegue

Asignatura	ITY1101 — Gestión de Datos para IA
Sección	003V
Proyecto	Telco Customer Churn — predicción de abandono de clientes
Integrantes	Benjamín Heresmann — Ingeniero de Datos Diego Hernández — Ingeniero de BD / DataOps
Metodología	PMBOK híbrido (predictivo + adaptativo)
Entrega	Julio 2026

Índice

1. Resumen ejecutivo y valor para la organización
 2. Metodología PMBOK y plan de gestión del proyecto
 3. Desarrollo del proyecto — Fase 1: pipeline DataOps
 4. Desarrollo del proyecto — Fase 2: modelo de IA y análisis de rendimiento
 5. Requerimiento de infraestructura de la solución
 6. Estrategia de monitoreo
 7. Protocolos de seguridad y gobernanza
 8. Estrategia de despliegue organizacional y beneficios
 9. Hallazgos, limitaciones y plan de mejoras
 10. Conclusiones
- Anexo A — Evidencias y recursos

1. Resumen ejecutivo y valor para la organización

Una telco pierde el **26,5%** de sus clientes al año y captar un cliente nuevo cuesta entre 5 y 7 veces más que retener a uno existente. Este proyecto construye una **solución de datos e IA de extremo a extremo** que anticipa qué clientes van a abandonar (*churn*), para que el negocio actúe **antes** de perderlos: un **pipeline DataOps** que ingesta, limpia, valida y carga 7.043 registros a una base de datos en la nube (Fase 1), y un **modelo de clasificación supervisado** que predice el abandono con un **recall del 79.7%** —detecta 4 de cada 5 fugas—, servido como microservicios y consumido por un **dashboard de BI** con demo en vivo (Fase 2).

Valor para la organización: el modelo prioriza a los **2.914 clientes en riesgo** por probabilidad de fuga, transformando la retención de reactiva a **proactiva y focalizada**: el equipo comercial concentra sus incentivos en la fracción correcta de la cartera en lugar de campañas masivas. La solución es **reproducible** (pipeline idempotente, código versionado), **auditable** (logs por corrida, RLS) y **cumple la Ley 21.719** de protección de datos desde el diseño. Este informe agrega la vista de operación: **requerimiento de infraestructura, estrategia de monitoreo, protocolos de gobernanza y plan de despliegue organizacional**.

2. Metodología PMBOK y plan de gestión del proyecto

2.1 Justificación de la metodología

Se adoptó un enfoque **PMBOK híbrido**: ciclo de vida **predictivo** para la estructura del proyecto —el alcance, los entregables y las fechas de las evaluaciones estaban definidos desde el inicio, lo que exige planificación por hitos— combinado con **desarrollo adaptativo (sprints semanales)** dentro de cada fase, porque los requisitos técnicos evolucionaron con la retroalimentación del docente (p. ej., desacoplar el monolito en microservicios tras la Parcial 2, o separar entrenamiento e inferencia en la Parcial 3). Un ejemplo concreto de esta alineación: el hito «Parcial 3» fijaba el entregable (modelo + dashboard), pero el *cómo* se iteró en sprints —primero el modelo baseline, luego el balanceado, luego el serving—. PMBOK aporta control de alcance y riesgos; lo adaptativo, capacidad de reacción.

2.2 Plan de gestión

Grupo de procesos PMBOK	Aplicación concreta en el proyecto
Inicio	Caso de negocio: costo del churn; acta implícita del encargo; selección del caso Telco (7.043 registros reales)
Planificación	EDT por fases (pipeline → modelo → operación); hitos =

Grupo de procesos PMBOK	Aplicación concreta en el proyecto
	evaluaciones; arquitectura cloud objetivo; matriz de riesgos
Ejecución	Sprints semanales por etapa del pipeline y del modelo; pair-review entre integrantes; commits descriptivos
Monitoreo y control	CI con pytest en cada push (GitHub Actions); revisión del docente por parcial como control de calidad; gestión de cambios vía issues/commits
Cierre	EFT: informe consolidado, defensa con demo y lecciones aprendidas

Herramientas y evidencias: el repositorio **GitHub** operó como herramienta de gestión —el historial de commits documenta los sprints, las ramas los cambios mayores y los releases los hitos— y **GitHub Actions** como control automatizado. **Gestión de riesgos** aplicada: se identificaron y mitigaron riesgos reales, como la suspensión del **free tier** por inactividad (mitigación: verificación previa a cada demo), el sobreajuste del modelo (mitigación: holdout estratificado) y la fuga de credenciales (mitigación: `.env` + escaneo del historial). En un despliegue organizacional, esta gestión escalaría a **Jira o MS Project** con tableros por equipo.

3. Desarrollo del proyecto — Fase 1: pipeline DataOps

Necesidad de DataOps frente al enfoque tradicional: en un flujo tradicional, un analista procesa un CSV a mano, sin versionado ni validación reproducible: cada corrida es distinta, los errores se descubren tarde y nada es auditable. **DataOps** aplica principios de ingeniería (automatización, testing, versionado, monitoreo) al ciclo de vida del dato. En este proyecto, eso se materializa en un **pipeline de cuatro etapas —ingesta → limpieza y transformación → validación estructural y semántica → carga—** totalmente automatizado y reproducible: la misma entrada produce siempre la misma salida (carga **full-refresh** idempotente y transaccional).

El pipeline procesa **7.043 registros** y los deja **con 0 nulos y tipos correctos** en **PostgreSQL 17 (Supabase)**; los registros que fallan la validación no se pierden: quedan auditados en `clientes_rechazados` con su motivo. Cada corrida registra su evidencia en `carga_logs` (leídos, insertados, rechazados, duración, estado). El procesamiento se expone como **API REST** (FastAPI + Swagger) desplegada en **Railway**, cada etapa corre como **un contenedor por capa** (Docker), y el código se versiona en **GitHub con CI** (pytest en cada push). Las **mejoras de la**

Parcial 2 ya están incorporadas: arquitectura desacoplada (GitHub · Railway · Supabase), modularización por contenedores, Row-Level Security y métricas por etapa en la API.

4. Desarrollo — Fase 2: modelo de IA y análisis de rendimiento

4.1 Diseño y entrenamiento

Sobre los datos curados se entrena un **clasificador binario supervisado** (objetivo: churn). Como la clase está **desbalanceada** (26,5% de churn), se prioriza el **recall** —en retención, el error caro es el **falso negativo**: un cliente que se va sin ser detectado— y se maneja el desbalance con `class_weight='balanced'` (SMOTE fue evaluado y descartado para no introducir datos sintéticos). Partición **70/30 estratificada**: train 4.930 / test 2.113. El preprocesamiento (estandarización de numéricas + one-hot de categóricas) vive **dentro** del Pipeline de scikit-learn, evitando fuga de datos y garantizando que producción aplique las mismas transformaciones que el entrenamiento.

Modelo	Accuracy	Recall	Precision	F1	Gini
LogReg (baseline)	81.0%	56.0%	67.1%	0.610	0.693
Arbol (baseline)	79.5%	60.6%	61.6%	0.611	0.664
LogReg balanceada	74.2%	79.7%	50.9%	0.621	0.693
RandomForest balanceado	77.1%	63.5%	56.1%	0.596	0.645

Se selecciona la **Regresión Logística balanceada**: mejor F1 (0.621) y mayor recall (79.7%). Su exactitud (74,2%) es menor que la del baseline (81,0%), pero esa exactitud es **engañosa**: por el desbalance, predecir «nadie se va» ya daría ~73,5% sin detectar un solo abandono. El Random Forest ilustra el **sobreajuste**: 100% de recall en entrenamiento cae a 63,5% en prueba; la Regresión Logística rinde casi igual en ambos conjuntos.

4.2 Análisis de rendimiento del modelo

La **matriz de confusión** sobre los 2.113 clientes de prueba: de los 561 que realmente abandonan, el modelo detecta **447 (TP)** y deja escapar **114 (FN)**; acierta **1.121 (TN)** de los que se quedan y genera **431 falsas alarmas (FP)**. El **recall 79.7%** = $TP/(TP+FN)$ captura 4 de cada 5 fugas; la **precisión 50.9%** refleja el costo en falsas alarmas (aceptable: una falsa alarma solo gasta un incentivo); el **F1 0.621** equilibra ambos. La **curva ROC** entrega un **AUC de 0.846**, es

decir un **coeficiente de Gini de 0.693** ($\text{Gini} = 2 \cdot \text{AUC} - 1$): el modelo ordena por riesgo con una capacidad muy superior al azar ($\text{Gini} = 0$), lo que habilita priorizar campañas por probabilidad.

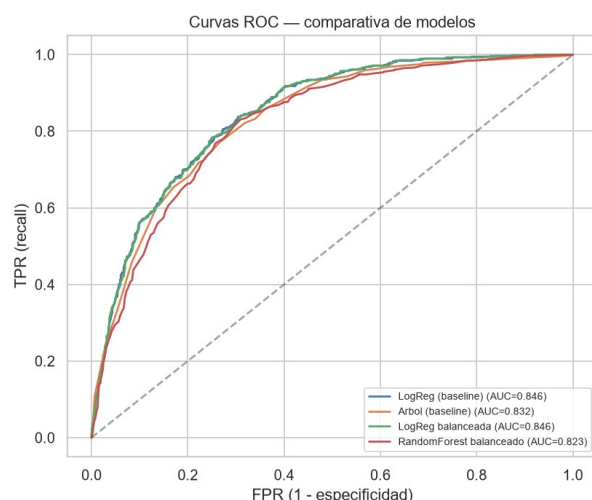


Figura 1. Curvas ROC de los cuatro modelos evaluados (AUC del elegido: 0.846).

4.3 Serving e integración BI

El modelo se opera como **dos microservicios independientes** (misma imagen, rol por variable de entorno) que se comunican solo vía Supabase: **trainer** (POST /train) entrena y guarda el modelo serializado en modelo_artefacto; **predictor** (GET /metrics, POST /predict/batch, GET /predict/cliente/{id}, POST /predict/nuevo) lo carga y predice **sin re-entrenar**. El endpoint /predict/nuevo predice el riesgo de un **cliente que el modelo nunca vio** y explica los **factores** que empujan la predicción (contribución coeficiente $\times$ valor), demostrando **generalización e interpretabilidad**. Un **dashboard Streamlit + Plotly** consume la tabla predicciones como fuente única: KPIs, matriz de confusión, churn por segmento, análisis de errores caso a caso, embudo del pipeline y formulario de cliente nuevo — todo demostrable en vivo.

5. Requerimiento de infraestructura de la solución

Decisión nube vs. on-premise: se requiere **nube** como plataforma principal. Justificación: (1) **costo inicial cero** frente a adquirir y operar servidores propios; (2) **elasticidad** —los picos de cómputo son breves (entrenamiento: 1,3 s) y no justifican hardware dedicado—; (3) **servicios gestionados** (PostgreSQL con backups y TLS incluidos) que reducen carga operacional. *On-premise* solo se justificaría por una política estricta de soberanía de datos; la Ley 21.719 no lo exige si el tratamiento cumple sus garantías. La implementación actual ya opera 100% en nube (Railway + Supabase + GitHub), validando la decisión.

5.1 Recursos técnicos requeridos (producción organizacional)

Recurso	Requerimiento	Justificación
Cómputo — API y predictor	2 servicios de 1–2 vCPU / 2–4 GB RAM c/u	Medición real: ~200 MB RAM y CPU ~10% para 7.043 registros; margen 10× para crecer
Cómputo — trainer	Job bajo demanda (no 24/7), 2 vCPU / 4 GB	Entrena en segundos; pagar solo al ejecutar
Almacenamiento	PostgreSQL gestionado 50 GB, backups diarios, retención 30 días	Base actual < 10 MB; margen para historial de predicciones y nuevas fuentes
Red	TLS 1.2+ en todo el tráfico; servicios y BD en la misma región	El benchmark mostró que la latencia de red domina (~218.8× vs. local): colocalizar reduce el cuello de botella
Disponibilidad	Health checks + reinicio automático; ≥ 2 réplicas del predictor tras balanceador (objetivo 99,9%)	El predictor es sin estado (el modelo vive en la BD): replicar es trivial
Redundancia	Réplica de lectura de la BD + backups verificados	La BD es el único punto único de fallo real del diseño

Software y dependencias: Python 3.11, scikit-learn 1.9 (versión **fijada**: el artefacto serializado exige la misma versión al deserializar), FastAPI/uvicorn, Streamlit, PostgreSQL 17, Docker. **Escalabilidad:** el diseño escala horizontalmente —el predictor es **stateless** y el pipeline procesa por lotes—; para volúmenes de millones de registros, la evolución natural es particionar la carga por lotes e incorporar un orquestador (Airflow) manteniendo la misma lógica por etapas.

6. Estrategia de monitoreo

La estrategia cubre **tres planos**: infraestructura (¿está viva la solución?), datos (¿el pipeline procesa bien?) y modelo (¿sigue prediciendo bien?). Se distingue lo **ya implementado** de la **propuesta de evolución**.

6.1 Implementado hoy

(1) **Salud de servicios**: cada microservicio expone GET /health; Railway lo consulta como *health check* y aplica **reinicio automático** ante fallo (restartPolicyType: ON_FAILURE). (2) **Auditoría del pipeline**: cada corrida registra en carga_logs los registros leídos/insertados/rechazados, duración y estado — un historial consultable de la salud del dato. (3) **Rendimiento**: medición instrumentada con psutil (tiempo por operación, RAM, CPU) documentada en outputs/rendimiento/. (4) **Visibilidad en vivo**: el dashboard se auto-refresca y refleja el estado real de la BD (modelo entrenado, predicciones) cada 3 segundos.

6.2 Propuesta de evolución

Plano	Métrica clave	Herramienta propuesta	Alerta proactiva
Infraestructura	Latencia p95, tasa de error 5xx, RAM/CPU por servicio, uptime	Prometheus + Grafana (exporters en cada servicio)	Slack/correo si p95 > 2 s o error rate > 1%
Datos	% de registros rechazados por corrida, duración del pipeline	Consulta programada sobre carga_logs	Alerta si rechazados > 5% o duración > 2× histórico
Modelo	Deriva de datos (PSI por variable), recall mensual re-etiquetado	Job programado de evaluación contra churn real observado	Alerta si recall < 70% → disparar re-entrenamiento

La cadena **Jenkins** del material se cubre en este proyecto con **GitHub Actions** (mismo rol: CI que ejecuta pytest en cada push); Prometheus/Grafana quedan propuestos porque el valor de monitoreo continuo aparece con operación 24/7 organizacional. El principio rector: **detección de incidencias antes que el usuario** — la alerta de deriva del modelo es la más importante, porque un modelo degradado falla en silencio.

7. Protocolos de seguridad y gobernanza

7.1 Controles de seguridad implementados (con mejoras de la Parcial 3)

(1) **Credenciales:** 0 secretos en el código y en el historial de git (verificado); las claves viven en variables de entorno y .env está en .gitignore. (2) **Acceso a datos:** Row-Level Security activo en todas las tablas, cerradas por defecto; los roles públicos anon/authenticated no leen nada; existe un rol de **solo lectura** (telco_lectura) para consumo analítico — privilegio mínimo. (3) **Cifrado:** TLS en tránsito en toda la cadena (API, BD, dashboard). (4) **Entorno:** escaneo de dependencias con pip-audit (10 CVEs detectados y con plan de actualización) y propuesta de trivy para las imágenes. (5) **Auditoría:** carga_logs + historial de git + esta auditoría documentada como evidencia.

7.2 Gobernanza de datos

Componente de gobernanza	Definición en el proyecto
Clasificación de datos	Datos personales no sensibles (género, edad, servicios, cargos); customerID seudonimizado (sin nombre/RUT)
Linaje del dato	Trazable por zonas: raw → clean → validated → BD; cada archivo con timestamp de corrida
Políticas de acceso	Dueño (pipeline, escritura) / telco_lectura (BI, lectura) / anon (bloqueado) — separación de funciones
Versionado del modelo	modelo_artefacto guarda modelo + métricas + fecha: qué modelo estaba en producción y cuándo
Retención y derechos	Indexación por customer_id habilita derechos ARCO (acceso, rectificación, cancelación) de la Ley 21.719
Gestión de riesgos	Matriz: fuga de credenciales (mitigada), CVEs (plan), free tier (keep-alive), deriva del modelo (monitoreo §6)

Cumplimiento normativo: la **Ley 21.719** (moderniza la 19.628; vigencia plena 01-12-2026, estándar tipo GDPR) se cumple **desde el diseño:** finalidad única (predecir churn), minimización (solo variables necesarias, sin identificadores directos), seguridad (controles anteriores) y responsabilidad proactiva (evidencia auditable). La gobernanza no es un documento aparte: **está embebida en el sistema** (RLS, logs, zonas, versionado).

8. Estrategia de despliegue organizacional y beneficios

El despliegue en una organización real se plantea como **adopción progresiva en cuatro fases**, minimizando el impacto operacional y generando confianza con resultados tempranos:

Fase	Alcance	Criterio para avanzar
1. Piloto (4–6 semanas)	Equipo de retención usa el dashboard (solo lectura) sobre la cartera; contacto manual del top-100 en riesgo	Tasa de retención del grupo contactado > grupo control
2. Integración CRM	El CRM consulta el predictor vía API (predicción individual en la ficha del cliente)	Adopción por ejecutivos; latencia < 1 s
3. Automatización	Scoring batch semanal alimenta campañas segmentadas automáticas	ROI de campañas > costo de incentivos
4. Operación continua	Re-entrenamiento programado + monitoreo de deriva (§6) + comité de gobernanza	Recall estable $\geq 75\%$ en producción

Integración con sistemas existentes: la API REST es el contrato — el CRM consume POST /predict/nuevo (predicción al vuelo con factores explicativos, útil para el ejecutivo en llamada) y el batch semanal actualiza predicciones para los tableros. **Migración de datos:** el pipeline ya acepta la fuente por archivo; conectarlo al sistema transaccional real solo cambia la etapa de ingesta (el resto del pipeline no se toca). **Capacitación:** sesiones cortas por rol (ejecutivos: leer el dashboard y el porqué de cada predicción; analistas: interpretar métricas; TI: operación y alertas). **Actualizaciones sin riesgo:** el versionado del modelo en `modelo_artefacto` permite *rollback* inmediato al modelo anterior si una versión nueva degrada.

Beneficios esperados: con recall del 79.7%, de cada 100 fugas el sistema anticipa ~80; incluso reteniendo solo una fracción de ellas, el ahorro supera con holgura el costo de operación (la infraestructura del §5 es de bajo costo). Beneficios cualitativos: decisiones basadas en datos, trazabilidad ante auditorías y cumplimiento normativo demostrable.

9. Hallazgos, limitaciones y plan de mejoras

Limitaciones detectadas **con evidencia**, la mejora que deriva de cada una y cómo se implementaría:

Limitación (evidencia)	Mejora propuesta	Implementación
Precisión 50,9%: 1 de 2	Ajustar umbral de decisión /	Barrido del umbral sobre el

Limitación (evidencia)	Mejora propuesta	Implementación
alarmas es falsa	priorizar por probabilidad	holdout optimizando F1 o costo de negocio
Random Forest sobreajusta (recall 100% train → 63,5% test)	Regularización (max_depth, min_samples_leaf)	Búsqueda de hiperparámetros con validación cruzada
Dataset pequeño y desbalanceado (7.043; 26,5%)	Validación cruzada k-fold + feature engineering	cross_val_score (k=5) en el modo --eval del pipeline de modelado
Latencia de red domina (~218.8x vs. local)	Colocalizar cómputo y datos / cachear lecturas	Misma región Railway–Supabase; caché del artefacto ya implementada en el predictor
10 CVEs en dependencias	Actualización planificada	Renovar python-dotenv de inmediato; upgrade coordinado de FastAPI/starlette con re-test
Free tier se suspende por inactividad	Keep-alive o tier pago en producción	Ping programado a /health; en organización: plan de pago (\$5)

El **hallazgo transversal** de las mediciones: el costo real de la solución no está en el cómputo del modelo (segundos) sino en la **latencia de red** y en la **operación** (monitoreo, seguridad, gobernanza) — exactamente lo que este informe planifica en los §5–§8. Varias mejoras propuestas en parciales anteriores **ya fueron implementadas**: microservicios de entrenamiento/inferencia separados, rol de solo lectura y predicción sobre clientes no vistos con explicación de factores.

10. Conclusiones

El proyecto cierra el ciclo completo de una solución de datos e IA: desde el dato crudo hasta un modelo en producción con **recall del 79.7%**, dashboard de BI y demo reproducible en vivo, y ahora con su **plan de operación organizacional** (infraestructura dimensionada con mediciones reales, monitoreo en tres planos, gobernanza embebida y despliegue progresivo). La arquitectura desacoplada demostró su valor: cada mejora (microservicios, predicción de clientes nuevos, endurecimiento de seguridad) se incorporó **sin rehacer** lo anterior.

Los aprendizajes clave: (1) en clases desbalanceadas, la **métrica correcta** (recall) vale más que la exactitud aparente; (2) la **operación** —monitoreo, seguridad, gobernanza— es tan determinante como el modelo para que la solución genere valor sostenido; y (3) medir antes de

dimensionar: el requerimiento de infraestructura de este informe se apoya en benchmarks reales, no en supuestos.

Anexo A — Evidencias y recursos

Recurso	Ubicación
Repositorio GitHub (código + CI)	github.com/BenjaminHeresmann/telco-churn-pipeline
API del pipeline (Swagger)	telco-api-production-e466.up.railway.app/docs
Microservicio trainer (Swagger)	telco-trainer-production.up.railway.app/docs
Microservicio predictor (Swagger)	telco-predictor-production.up.railway.app/docs
Dashboard BI en vivo	telco-dashboard-production.up.railway.app
Presentación (deck)	deck-benjaminheresmanns-projects.vercel.app
Modelo, métricas y auditoría	src/modelo.py · outputs/modelo/ · outputs/seguridad/
Informes previos (Fase 1 y 2)	docs/Informe_Tecnico_Evaluacion2.pdf · docs/Informe_Tecnico_Evaluacion3.pdf

Las cifras de este informe se generan automáticamente desde los artefactos reales del proyecto ([outputs/modelo/metricas_modelos.csv](#), [outputs/rendimiento/benchmark.json](#)), garantizando trazabilidad y reproducibilidad.